3.5 컴포넌트와 이벤트

3.5.1 이벤트 속성

- 이벤트(event): 사용자와의 상호작용에 따라 발생하는 일련의 사건
- (예) 마우스 버튼을 클릭하거나 키보드로 글자를 입력하는 등의 동작들
- 리액트에서도 이러한 이벤트를 활용해 사용자와 상호작용하는 기능을 구현 가능함

▶ 이벤트를 사용하는 기본 문법

형식

```
<JSXElement 이벤트_속성='이벤트_핸들러'/>
```

계속

3.5 컴포넌트와 이벤트

3.5.1 이벤트 속성

- HTML에서 이벤트 속성은 사용자가 특정 동작을 수행했을 때 ((예) 마우스 클릭, 키 입력 등) 실행할 JavaScript 코드를 지정하는 속성
- React 에서도 이벤트를 비슷한 방식으로 사용할 수 있지만 몇 가지 차이점이 있음
- HTML 에서는 이벤트 속성을 소문자로 작성하고, 값은 문자열 형태의 JavaScript 코드로 작성함
- React(JSX)에서는 이벤트 속성을 카멜 케이스((예) onClick, onSubmit)으로 작성하고, 값으로 함수를 전달함

계속

3.5 컴포넌트와 이벤트

3.5.1 이벤트 속성

▶ React 에서 버튼 클릭 이벤트를 처리하는 예제

```
export default function App() {
  return (
    <>
      <button
        style={{
            width: "300px",
            height: "200px"
          }}
        onClick=
        {
              () => { alert("Button Clicked!") }
        } >
        click
      </button>
    </>
  );
}
```

계속

3.5 컴포넌트와 이벤트

3.5.1 이벤트 속성

▶ 리액트의 주요 이벤트 속성

✓ 별표(*) : 자주 사용하는 이벤트 속성

이벤트 유형	이벤트 속성	설명
마우스 이벤트	onClick ★	클릭 이벤트 발생 시 호출
	onDoubleClick	더블 클릭 이벤트 발생 시 호출
	onMouseEnter	마우스가 요소 위로 올라갔을 때 호출
	onMouseLeave	마우스가 요소에서 이동했을 때 호출
	onMouseMove	마우스가 요소 위에서 움직일 때 호출

계속

3.5 컴포넌트와 이벤트

이벤트 유형	이벤트 속성	설명
마우스 이벤트	onContextMenu	마우스 오른쪽 버튼 클릭 시 호출
	onDrag	드래그할 때 호출
	onDrop	드래그한 요소를 놓을 때 호출
	onMouseDown	마우스 버튼을 눌렀을 때 호출
	onMouseUp	마우스 버튼을 눌렀다 뗐을 때 호출

이벤트 유형	이벤트 속성	설명
키보드 이벤트	onKeyDown ★	키를 눌렀을 때 호출
	onKeyUp ★	키를 눌렀다 뗐을 때 호출
	onKeyPress	키를 누르고 있을 때 호출

계속

3.5 컴포넌트와 이벤트

이벤트 유형	이벤트 속성	설명
폼 이벤트	onChange ★	입력 필드의 값이 변경될 때 호출
	onInput	입력 필드에서 입력이 발생할 때 호출
	onFocus	입력 필드에 포커스가 들어올 때 호출
	onBlur	입력 필드에서 포커스가 빠져나갈 때 호출
	onSubmit ★	폼이 제출될 때 호출
터치 이벤트	onTouchStart	터치가 시작될 때 호출
	onTouchMove	터치 중에 움직일 때 호출
	onTouchEnd	터치가 끝났을 때 호출
기타 이벤트	onLoad	이미지나 iframe 등이 로드될 때 호출
	onError	이미지나 iframe 등에서 오류가 발생했을 때 호출
	onScroll	요소가 스크롤될 때 호출

3.5 컴포넌트와 이벤트

●이벤트 속성 사용 시 주의사항

- JSX에서 HTML 태그에는 DOM 표준 이벤트 속성만 사용할 수 있음
- 아래 코드와 같이 존재하지 않는 속성(onFive)을 사용하면 웹 브라우저에서 인식할 수 없어 오류가 발생하거나 무시됨.

```
export default function App() {
  return (
    <>
      <button onFive={() => { alert('five'); }}>click</button>
    </>
  );
}
```

존재하지 않는 이벤트 속성 사용으로 오류 발생

3.5 컴포넌트와 이벤트

●이벤트 속성 사용 시 주의사항

- 하지만, 컴포넌트는 HTML 태그와 달리 직접 만든 사용자 정의 함수임.
  - 아래와 같이 원하는 이름의 이벤트 속성을 자유롭게 정의해 props로 전달 가능함.

src/App.tsx

```
import Button from "./components/Button";

export default function App() {
  return (
    <>
      <Button onFive={() => alert("five")} />
    </>
  );
}
```

●이벤트 속성 사용 시 주의사항

- 컴포넌트에서는 전달한 props 값을 받아 실제 DOM 요소에 연결할 수 있음
  - (예) Button 컴포넌트는 onFive를 props 로 받아 onClick 이벤트와 연결해 사용 가능함.

src/components/Button.tsx

```
export default function Button(
    { onFive }: { onFive: () => void })
{
    return (
        <button
                style={{
                    width: "300px",
                    height: "200px"
                }}  onClick={onFive}
                >
                    click
        </button>
    ) ;
}
```

✓ **함수 컴포넌트는 사용자 정의 속성(onFive)을 props 로 전달받아 DOM 요소의 표준 이벤트 속성(onClick)에 연결하는 구조로 이벤트를 유연하게 관리 가능함**

3.5 컴포넌트와 이벤트

3.5.2 이벤트 핸들러

- 이벤트 핸들러: 이벤트가 발생했을 때 실행할 함수
- React 에서는 JSX요소에 이벤트 속성을 사용해 이벤트를 등록하는데 이때 이벤트 속성에 이벤트 핸들러를 할당해야 함.
- 이벤트 핸들러를 작성하는 방식 두 가지

① 인라인 핸들러(inline handler)

- 이벤트 속성 안에 함수를 직접 작성하는 방식
  - (예) 버튼을 클릭했을 때 경고창을 띄우고 싶다면 다음과 같이 작성함

3.5 컴포넌트와 이벤트

3.5.2 이벤트 핸들러

① 인라인 핸들러(inline handler)

```
export default function App() {
  return (
    <>
      <button
        onClick= { () => { alert("Button Clicked!")} } > click </button>
    </>
  );
}
```

- ✓ 코드에서 onClick 이벤트 속성 안에 화살표 함수를 직접 작성함.
- ✓ 인라인 핸들러(inline handler) 방식은 이벤트 핸들러를 간단하게 할당할 수 있고 코드가 간결해 유용함.

3.5 컴포넌트와 이벤트

3.5.2 이벤트 핸들러

●함수 참조

- 이벤트 핸들러를 컴포넌트 바깥(JSX 바깥)에 따로 정의하고, 해당 함수를 참조해 이벤트 속성에 할당하는 방식
- 다음 코드는 JSX 외부에 clickHandler() 함수를 정의하고 onClick 속성에 함수 이름을 전달함. 버튼을 클릭했을 때만 함수가 실행됨.

3.5.2 이벤트 핸들러

●함수 참조

```
<button
        style={{
            width: "300px",
            height: "200px"
          }}

        onClick={ clickHandler }>click
</button>
```

- 함수 참조 방식을 사용할 때 함수의 매개변수가 전혀 없을 때는 함수명 뒤에 괄호를 붙이면 안됨.
- 괄호를 붙이면 함수가 바로 실행되므로 버튼을 클릭하지 않아도 컴포넌트가 화면에 나타나는 순간 함수가 실행됨
- 다음 코드는 잘못된 방식임.

3.5 컴포넌트와 이벤트

3.5.2 이벤트 핸들러

●함수 참조

```
<button
        style={{
            width: "300px",
            height: "200px"
          }}

        onClick={ clickHandler() }>click
</button>
```

✓ 괄호를 붙이면 함수가 바로 실행되므로 버튼을 클릭하지 않아도 컴포넌트가 화면에 나타나는 순간 함수가 실행됨.

3.5 컴포넌트와 이벤트

3.5.2 이벤트 핸들러

● 이벤트 핸들러의 매개변수

- React 에서는 이벤트 핸들러에 매개변수를 전달할 때 인라인 핸들러 방식을 사용해야 함.
- 함수 참조 방식으로는 매개변수를 전달할 수 없음.
- (예) 인라인 핸들러 방식으로 버튼을 클릭했을 때 만 “hello”라는 메시지를 출력하는 프로그램.

```
export default function App() {
  return (
    <>
      <button
        onClick= { () => { alert("Button Clicked!")} } > click </button>
    </>
  );
}
```

3.5 컴포넌트와 이벤트

3.5.2 이벤트 핸들러

● 이벤트 핸들러의 매개변수

- 함수 참조 방식으로는 매개변수를 전달하면 함수가 즉시 실행되는 문제가 발생함
- (예) 함수 참조 방식으로 코드를 작성하면 버튼을 클릭하지 않아도 컴포넌트가 렌더링될 때 clickHandler(“hello”) 가 즉시 실행되어 “hello” 라는 메시지를 출력하는 프로그램.

```
export default function App() {
  const clickHandler = (msg : string) => {
    alert(msg);
  };
  return (
    <>
        <button
          onClick={ clickHandler("hello") }>click
        </button>
    </>
  );
}
```

3.5.3 이벤트 객체

- React 에서는 이벤트가 발생하면 해당 이벤트 핸들러 함수가 호출되고, 이벤트 객체(event object)가 매개변수로 전달됨.
- 이때 전달되는 이벤트 객체는 웹 브라우저에서 제공하는 기본 DOM 이벤트 객체(native DOM event object)와는 다름
- React는 웹 브라우저의 기본 이벤트를 감싸서 만든 자체 이벤트 객체인 합성 이벤트(synthetic event)를 사용함.
  - 이는 웹 브라우저마다 이벤트 객체의 동작 방식이 조금씩 달라 이를 일관되게 처리하기 위해서임.
    - 이를 “크로스브라우저 호환성(cross-browser compatibility)을 확보한다“고 표현함.

3.5 컴포넌트와 이벤트

3.5.3 이벤트 객체

- 합성 이벤트는 기본 DOM 이벤트와 거의 동일한 속성과 메서드를 제공하므로 일반 DOM 이벤트처럼 사용 가능함.

- 이벤트 객체와 인라인 핸들러
  - 인라인 핸들러에서는 이벤트 객체를 함수의 매개변수로 직접 받아 사용 가능함.
  - (예) <button>요소에서 클릭 이벤트가 발생했을 때 이벤트 객체를 받아 콘솔에 출력함

```
export default function App() {
  return (
    <>
      <button
        onClick= { (e) => { console.log(e); } } > click </button>
    </>
  );
}
```

3.5.3 이벤트 객체

●이벤트 객체와 인라인 핸들러

- 코드에서 e는 이벤트 객체를 의미함. 보통 e 또는 event 라는 이름으로 자주 사용함.
- React 는 타입스크립트 환경에서도 이벤트 객체의 타입을 자동으로 추론하므로 타입을 명시하지 않아도 기본적인 속성과 메서드를 사용 가능함.
  - (예)e.target.value나 e.preventDefault()와 같은 메서드는 타입을 생략해도 문제없이 작동함.

3.5 컴포넌트와 이벤트

●이벤트 객체와 함수 참조

- 함수 참조 방식에서는 이벤트 핸들러에 이벤트 객체를 직접 전달하지 않아도 React가 자동으로 이벤트 객체를 해당 함수의 첫 번째 매개변수로 전달함.

3.5 컴포넌트와 이벤트

●이벤트 객체와 함수 참조

- (예) <button>요소에 clickHandler() 함수를 참조 형태로 전달할 때 이벤트 객체를 명시하지 않지만 해당 함수의 첫번째 매개변수로 e를 선언해 자동으로 이벤트 객체가 전달됨.

```
export default function App() {
  const clickHandler = (
    e: React.MouseEvent<HTMLButtonElement, MouseEvent>,
    msg: string
  ) => {
    console.log(e);
    alert(msg);
  };
  return (
    <>
      <button onClick={(e) => clickHandler(e, "Hello")}>click</button>
      { /*<button onClick={(e) => e }>click</button> */ }
    </>
  );
}
```

3.5 컴포넌트와 이벤트

●이벤트 객체와 함수 참조

- 함수 참조 방식에서 이벤트 객체를 사용하려면 참조 함수에 타입을 명시해야 함.
- 이벤트 객체의 타입은 JSX 요소와 이벤트 종류에 따라 달라지므로 정확히 외우기 어려움.
  - (해결 방법) 타입추론을 활용하면 됨.

❖ 타입 추론(Type Inference)

- 프로그래밍 언어에서 명시적으로 타입을 지정하지 않아도 컴파일이나 인터프리터가 변수의 타입을 자동으로 유추하는 기능을 말함

이벤트 객체와 함수 참조

- (해결 방법) 타입추론을 활용하면 됨.
  - 먼저 아래와 같이 인라인 핸들러 방식으로 코드를 작성함.
  - e 에 마우스를 올리면 자동으로 이벤트 객체의 타입을 추론해 보여줌.

```
ort default fur
```

```
(parameter) e:
React.MouseEvent<HTMLButtonElement,
MouseEvent>
```

```
<button onClick={(e) => e }>click</button>
```

  - 이렇게 확인한 타입을 함수 참조 방식으로 전환할 때 그대로 사용하면 됨.

3.5 컴포넌트와 이벤트

●이벤트 객체와 매개변수

- 이벤트 핸들러에 이벤트 객체와 추가 매개변수를 모두 전달해야 하는 경우
  - 인라인 핸들러 방식을 사용해 이벤트 객체와 원하는 매개변수를 명시적으로 전달해야 함

3.5 컴포넌트와 이벤트

●이벤트 객체와 매개변수

- 다음 코드는 onClick 이벤트 속성에 인라인 핸들러 방식으로 함수를 작성하고,이벤트 객체 e와 문자열 “Hello”를 함께 전달함.

```
export default function App() {
  const clickHandler = (
    e: React.MouseEvent<HTMLButtonElement, MouseEvent>,
    msg: string
  ) => {
    console.log(e);
    alert(msg);
  };
  return (
    <>
      <button onClick={(e) => clickHandler(e, "Hello")}>
          click</button>
    </>
  );
}
```

3.5 컴포넌트와 이벤트

●이벤트 객체와 매개변수

- 함수 참조 방식은 이벤트 객체 하나만 자동으로 전달 가능하므로 매개변수가 2개 이상이면 인라인 핸들러 방식으로 전달함
- 즉, 이벤트 핸들러에 이벤트 객체와 사용자가 원하는 값을 함께 전달해야 하는 경우
  - 인라인 핸들러 방식을 사용해야 함

3.5.4 이벤트 전파(event propagation)

- JSX 요소에서도 기존 DOM과 마찬가지로 이벤트 전파가 발생함
- 이벤트가 발생했을 때 부모 또는 자식 요소로 전달되는 현상

●이벤트 전파 기법 두가지

- 캡처링(capturing)과 버블링(bubbling)

① 캡처링(capturing)

- 이벤트가 부모 요소에서 시작해 자식 요소로 내려가는 방식
- React 에서 캡처링을 처리하려면 이벤트 속성에 Capture를 붙여 사용해야 함.

3.5 컴포넌트와 이벤트

●캡처링(capturing)

- (예) onClick 이벤트 속성에 Capture를 붙인 onClickCapture 이벤트 속성을 사용한 프로그램

```
export default function App() {
  const handleCapture = () => {
    console.log("Parent");
  };
  const handleBubble = (e: React.MouseEvent<HTMLButtonElement,
                                            MouseEvent>) => {
    e.stopPropagation();
    console.log("Child");
  };
  return (
    <div
      //onClick={handleCapture}
      onClickCapture={handleCapture}
      style={{ padding: "50px", backgroundColor: "#f0f0f0" }}
    >
      Parent
      <button onClick={handleBubble} style={{ marginTop: "20px" }}>
        Click Me
      </button>
    </div>
  );
}
```

3.5 컴포넌트와 이벤트

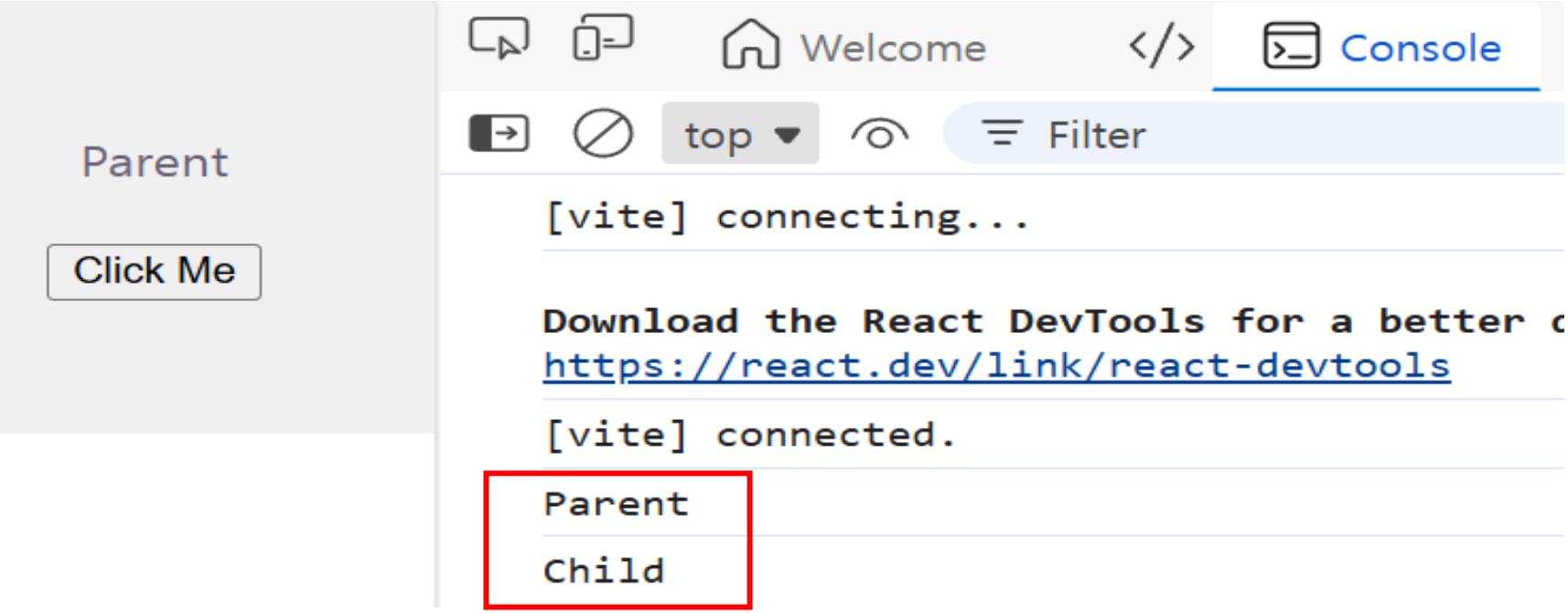
●캡처링(capturing)

- (예) onClick 이벤트 속성에 Capture를 붙인 onClickCapture 이벤트 속성을 사용한 프로그램
- 코드에서 <button> 요소를 클릭하면 onClick 이벤트가 발생함.
  - 이때 부모 요소인 에 onClickCapture 속성이 정의되어 이벤트가 자식 요소로 도달하기 전에 부모의 캡처링 핸들러가 먼저 실행됨.
- 즉, 이벤트는 부모 → 자식 방향으로 전파됨.

3.5 컴포넌트와 이벤트

●캡처링(capturing)

- 애플리케이션을 실행하고 버튼을 클릭하고 웹 브라우저의 콘솔창을 보면 Parent → Child 순으로 로그가 남음.
- <실행결과 화면>



② 버블링(bubbling)

- 이벤트가 자식 요소에서 시작해 부모 요소로 전파되는 방식
- (예)<button>요소에서 발생한 클릭 이벤트는 해당 버튼을 포함한 , <body>, <html>요소로 차례대로 전달됨.
- React에서는 버블링이 기본 동작이므로 캡처링처럼 Capture 수식어가 붙은 별도의 이벤트 속성이 필요하지 않고 이벤트 속성 이름을 그대로 사용하면 됨.

3.5 컴포넌트와 이벤트

② 버블링(bubbling)

- (예)

```
export default function App() {
  const handleCapture = () => {
    console.log("Parent");
  };
  const handleBubble = () => {
    console.log("Child");
  };
  return (
    <div
      onClick={handleCapture}
      style={{ padding: "20px",
               backgroundColor: "#ccc" }}
    >
      Parent
      <button onClick={handleBubble}
               style={{ marginTop: "20px" }}>
        Click Me
      </button>
    </div>
  );
}
```

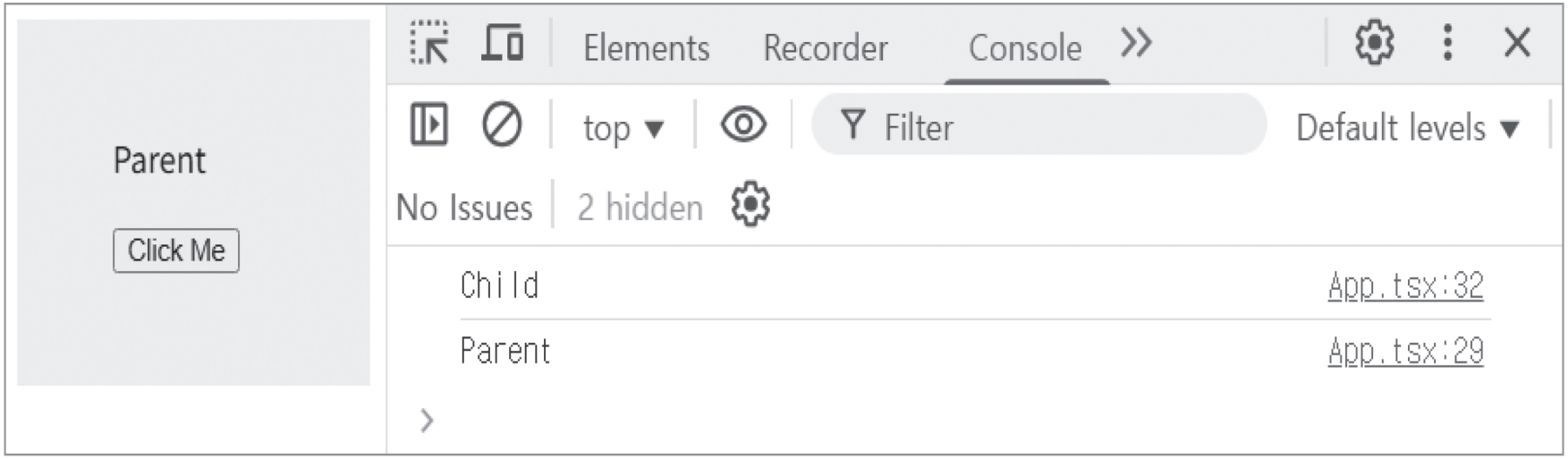
onClick 이벤트 속성에 Capture 수식어만 없음

3.5 컴포넌트와 이벤트

② 버블링(bubbling)

- 애플리케이션을 실행하고 버튼을 클릭하면 Child → Parent 순으로 로그가 출력됨

▶ 버블링 결과



3.5 컴포넌트와 이벤트

●이벤트 전파 막기

- 캡처링이 일어나지 않게 하려면 이벤트 속성에 Capture 수식어를 사용하지 않으면 됨.
- 하지만, 버블링은 기본 동작이므로 버블링을 막으려면 명시적으로 코드를 작성해야 함.
- 버블링을 차단하려면 이벤트 객체의 stopPropagation() 메서드를 사용해 버블링을 차단
  - 이 메서드는 이벤트가 상위 요소를 전파되는 것을 막아줌.

●이벤트 전파 막기

- (예) 버튼 클릭 시 stopPropagation() 메서드를 호출해 이벤트가 부모 요소로 전달되지 않도록 함

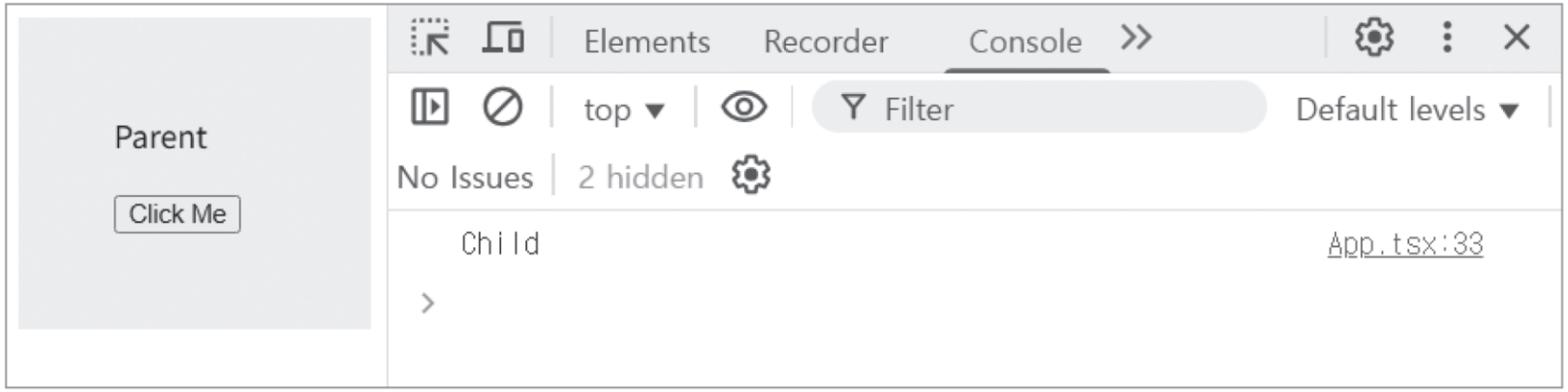
```
export default function App() {
  const handleCapture = () => {
    console.log("Parent");
  };
  const handleBubble = (e: React.MouseEvent<HTMLButtonElement,
                                            MouseEvent>) => {
    e.stopPropagation();
    console.log("Child");
  };
  return (
    <div
      onClick={handleCapture}
      style={{ padding: "50px", backgroundColor: "#f0f0f0" }}
    >
      Parent
      <button onClick={handleBubble} style={{ marginTop: "20px" }}>
        Click Me
      </button>
    </div>
  );
}
```

3.5 컴포넌트와 이벤트

●이벤트 전파 막기

- 이벤트 객체의 stopPropagation() 메서드를 사용해 버블링을 차단

▶ stopPropagation() 메서드 호출 결과

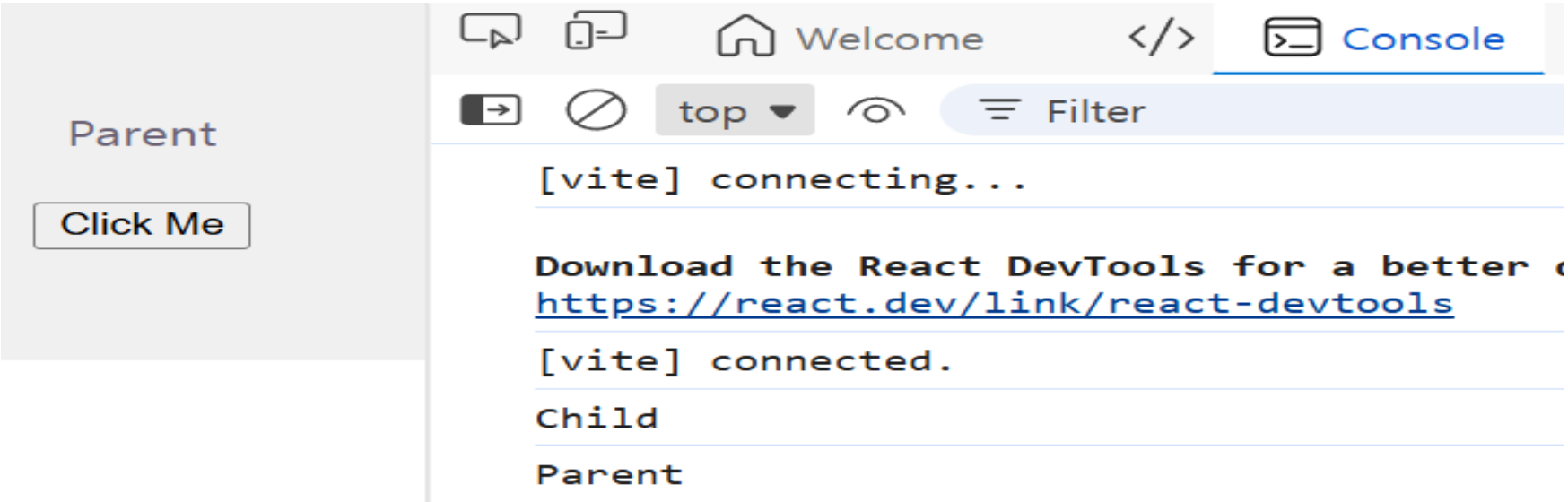


3.5 컴포넌트와 이벤트

●이벤트 전파 막기

- 캡처링이 일어나지 않게 하기 위해서 이벤트 속성에 Capture 수식어를 사용하지 않음.
  - 결과적으로 버블링으로 실행된 결과인 Child --> Parent 순으로 전파되어 출력됨

▶ 이벤트 속성에 Capture 수식어를 사용하지 않은 결과

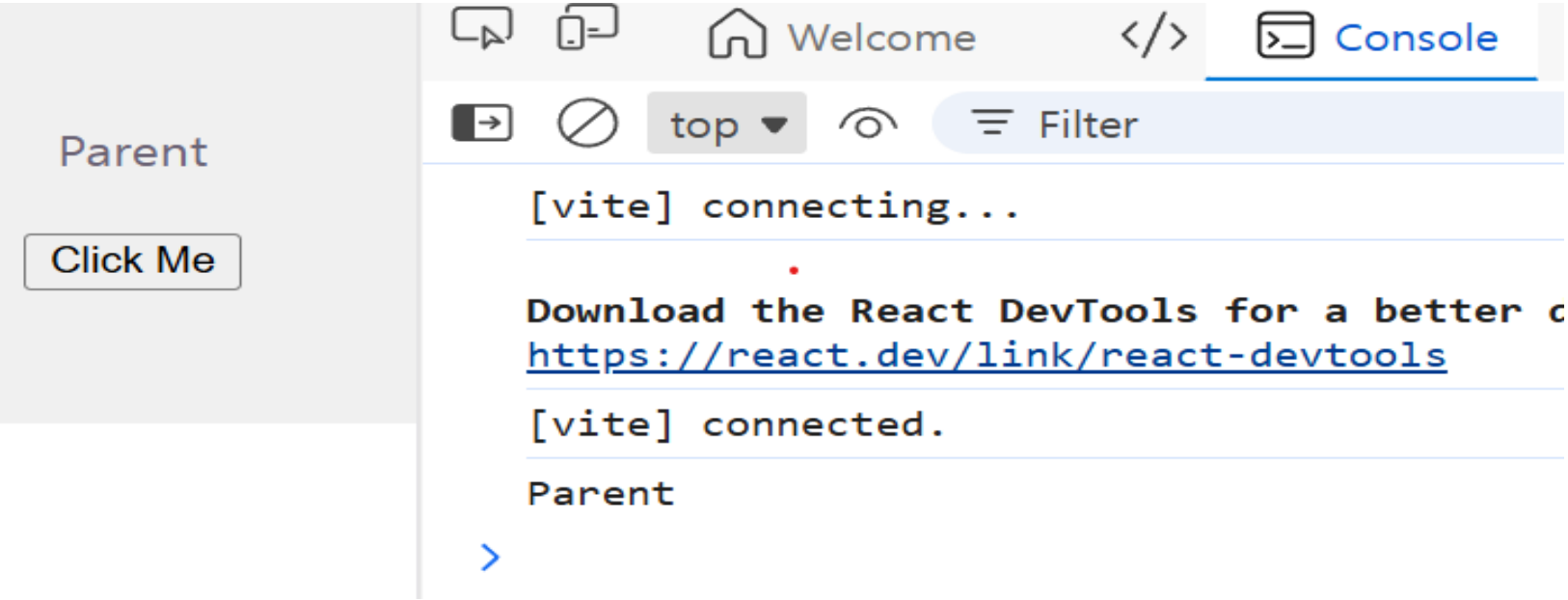


3.5 컴포넌트와 이벤트

●이벤트 전파 막기

- 캡처링을 차단하기 위해 버튼 클릭 시 Child → Parent(버블링)으로 실행되므로 Parent만 나오게 하려면 Child 실행을 막아야 함.
- (해결) Child 인 버튼의 onClick 이벤트 속성 자체를 제거하면 됨.

▶ Child 인 버튼의 onClick 이벤트 속성 자체를 제거한 결과



3.5 컴포넌트와 이벤트

●이벤트 전파 막기

- 캡처링을 차단하기 위해 "캡처링 차단"이라는 조건을 정확히 반영하여 캡처링을 사용하면서 Parent만 실행하는 코드
- 즉, 캡처링 이후 전파(버블링 포함)를 차단한 것

▶ 캡처링을 사용하면서 Parent만 실행하는 코드의 결과

3.5 컴포넌트와 이벤트

●이벤트 기본 동작 막기

- HTML 요소는 특정 이벤트가 발생하면 자동으로 실행하는 기본 동작이 있음.
- JSX 요소도 HTML 태그와 동일하게 이러한 기본 동작을 따름
  - (예) <a>태그는 클릭하면 링크로 이동하고, <button>태그는 클릭 이벤트를 발생시키며, <form>태그는 입력한 데이터를 서버로 제출하면서 페이지를 새로 고침.
- 하지만, React 애플리케이션에서는 이런 기본 동작을 막아야 할 상황이 자주 발생함.
  - (예) REST API를 통해 데이터를 비동기 방식으로 전송하는 경우에 <form>의 기본 동작(페이지 새로 고침)이 실행되면 React 애플리케이션의 상태와 UI가 초기화 되어 사용자가 입력한 데이터가 모두 사라질 수 있음.

3.5 컴포넌트와 이벤트

●이벤트 기본 동작 막기

- 이벤트 객체의 preventDefault() 메서드를 사용해 기본 동작 막을 수 있으며 이 메서드는 브라우저의 기본 동작을 취소함.
- (예)

```
export default function App() {
  return (
    <form
      onSubmit={(e) => {
        e.preventDefault();
      }}
    >
      <input type="text" name="email" />
      <input type="password" />
      <button type="submit">전송</button>
    </form>
  );
}
```

3.5 컴포넌트와 이벤트

●이벤트 기본 동작 막기

- <form>요소는 사용자가 입력한 데이터를 서버로 제출하는 기능을 기본으로 수행하고, 전송 후 자동으로 페이지를 새로 고침.
- onSubmit 화살표 함수는 폼이 제출될 때 자동으로 실행하는 이벤트 핸들러 함수
  - 이 함수에서 e.preventDefault()가 호출되어 페이지 새로 고침을 막음.
    - 단, preventDefault() 함수는 폼 제출시 기본 동작을 취소할 뿐, 버튼을 클릭하는 동작 자체는 막지 않음.
- <button type="submit">전송 </button>에서 “전송” 버튼을 클릭하면 <form>이 제출되는 submit 이벤트를 발생시킴.